

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

1. Основи на Python. Въведение в програмирането

Програмата на Python се състои от набор от инструкции. Всяка инструкция се поставя на нов ред.

Пример:

```
print(2+3)
print("Hello")
```

Резултат:

5

Hello

- **Отстъпът играе важна роля в Python.** Неправилният отстъп всъщност е грешка.

Пример:

В следния случай ще получим грешка, въпреки че кодът е почти същият като горния:

```
print(2+3)
print("Hello") File "<string>", line 5
```

Резултат:

```
print("Hello")
^
```

IndentationError: unexpected indent

Затова трябва да се поставят новите инструкции в началото на реда. Това е една от важните разлики между Python и другите езици за програмиране като C# или Java. Трябва обаче да се има предвид, че някои езикови конструкции могат да се състоят от няколко реда.

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Пример:

Условен оператор if:

```
if 1<2  
    print("Hello")
```

В този случай, ако 1 е по-малко от 2, тогава се показва низът „Hello“. И тук вече трябва да има отстъп, тъй като операторът `print("Hello")` не се използва самостоятелно, а като част от условната конструкция `if`. Освен това, отстъпът, според ръководството за проектиране на код: <https://peps.python.org/pep-0008/>, е желателно да се направи от такъв брой интервали, който е кратен на 4 (т.е. 4, 8, 16 и т.н.) Въпреки че, ако отстъпът не е 4, но 5, тогава програмата също ще работи. Няма толкова много такива конструкции, така че не трябва да има голямо объркване, относно поставянето на интервали.

- Python различава главни от малки букви

Python е език, чувствителен към малки и големи букви, така че изразите `print` и `Print`/или `PRINT` са различни. Ако вместо действителния метод за печат за извеждане в конзолата, се въведе метода записан като `Print`:

`Print("Hello World")` - тогава е **неправилно**.

2. Коментари

Коментарите се използват за маркиране на това, което прави част от кода. Когато дадена програма е преведена и изпълнена, интерпретаторът игнорира коментарите, така че те нямат ефект върху работата на програмата. Коментарите в Python биват: блокови и вградени коментари. **Коментарите на ред са предшествани от знака диез - #. Те могат да бъдат на отделен ред. Пример:**

```
# Вход в конзолата
```

```
# Съобщение Hello World
```

```
print("Hello World")
```

Всеки набор от знаци след знака # представлява коментар, т.е. първите два реда код са коментари. Коментарите могат да се поставят и на същия ред като езиковите инструкции, след като инструкциите бъдат изпълнени. **Пример:**

```
print("Hello World") # Изходно съобщение в конзолата
```

Блоковите коментари се поставят в три единични кавички преди и след текста на коментара: '''текст коментар'''. Пример:

```
'''
```

```
    Изход в конзолата
```

```
    Съобщение Hello World
```

```
'''
```

```
print("Hello World")
```

3. Основни функции

Python предоставя редица вградени функции. Някои от тях се използват много често, особено в началните етапи на изучаване на език.

1) Основната функция за извеждане на информация към конзолата е функцията `print()` . Низът, който ще се изведе, се предава като аргумент на тази функция:

```
print("Hello Python")
```

Ако трябва да се отпечатаат няколко стойности на конзолата, тогава те могат да се предадат на функцията за печат, разделени със запетаи:

```
print("Full name:", "Tom", "Smith")
```

В резултат на това всички предадени стойности ще бъдат залепени заедно с интервали в един ред:

Резултат: Full name: Tom Smith

2) Докато функцията за печат е отговорна за изхода, то функцията за въвеждане е отговорна за въвеждането на информация. Тази функция е поканена за въвеждане като незадължителен параметър и връща входния низ, който може да се съхрани в променлива:

```
name = input("Въведи име: ")
```

```
print("Здравей, ", name)
```

Резултат:

Въведи име: Здравей, Евгени

4. Променливи и типове данни. Променливи

Променливите са предназначени за съхранение на данни. **Името на променлива в Python трябва да започва с азбучен знак или долна черта и може да съдържа буквено-цифрови знаци и долна черта.** Освен това името на променливата не трябва да съвпада с ключова дума на езика Python.

5. Ключови думи

1	False	await	else	import	pass
2	None	break	except	in	raise
3	True	class	finally	is	return
4	and	continue	for	lambda	try
5	as	def	from	nonlocal	while
6	assert	del	global	not	with
7	async	elif	if	or	yield

Фиг.1 Ключови думи в Python

Пример: Дефинирайте променлива:

name="Tom"

Тук е дефинирана променлива, name, която съхранява низа "Tom". Python използва два типа именуване на променливи: **Camel** и нотация за долна черта. **Camel означава, че всяка нова поддума в името на променливата започва с главна буква.**

Пример: userName="Tom"

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Подчертаването означава, че поддумите в името на променливата са разделени с долна черта.

Пример:

```
user_name="Tom"
```

Освен това трябва да се вземе предвид **чувствителността на малки и големи букви**, така че променливите name и Name са различни обекти.

```
# две различни променливи
```

```
name = "Tom"
```

```
Name = "Tom"
```

След като се дефинира променлива, тя може да се използва в програмата.

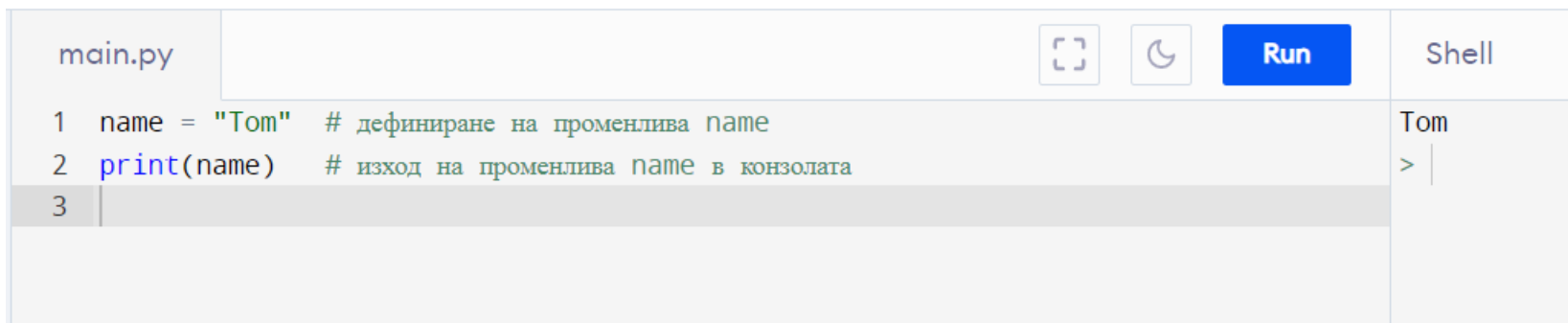
Пример:

Нека се отпечата съдържанието на променливата в конзолата с помощта на вградената функция print:

```
name = "Tom" # дефиниране на променлива name
```

```
print(name) # изход на променлива name в конзолата
```

Дефиниране и прилагане на променлива в Python:



The screenshot shows a code editor with a file named 'main.py'. The code contains two lines: '1 name = "Tom" # дефиниране на променлива name' and '2 print(name) # изход на променлива name в конзолата'. A third line is empty. To the right of the code editor is a 'Run' button and a 'Shell' window. The 'Shell' window displays the output 'Tom' and a prompt '> |'.

```
main.py  [ ] [ ] Run Shell
1 name = "Tom" # дефиниране на променлива name
2 print(name) # изход на променлива name в конзолата
3
```

Tom
> |

Фиг.2 Дефиниране на променлива в Python

Отличителна черта на **променливата е, че може да се промени стойността и по време на изпълнение на програмата:**

```
name = "Tom"      # променлива name приема стойност "Tom"
print(name)       # изход: Tom
name = "Bob"      # променливата name приема нова стойност "Bob"
print(name)       # изход: Bob
```

6. Типове данни

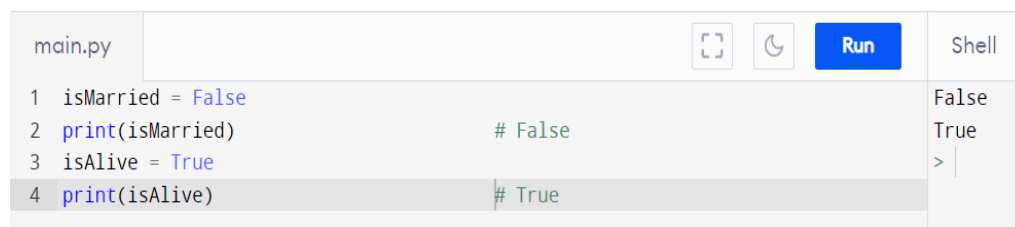
Променливата съхранява данни от един от типовете данни. Има много различни типове данни в Python. В този случай ще бъдат разгледани само най-основните типове: bool , int , float , complex и str .

6.1. Булеви стойности

Типът **bool** представлява две булеви стойности: True (true, true) или False (false, false). Стойността True се използва, за да посочи, че нещо е вярно. Докато стойността False, напротив, показва, че нещо е грешно.

Пример:

```
isMarried = False
print(isMarried)
isAlive = True
print(isAlive)
```



```
main.py
1 isMarried = False
2 print(isMarried)           # False
3 isAlive = True
4 print(isAlive)             # True
```

Run Shell

False
True
> |

Фиг.3 Екранна форма на резултат

6.2. Цели числа

Типът **int** представлява цяло число, като 1, 4, 8, 50. **Пример:**

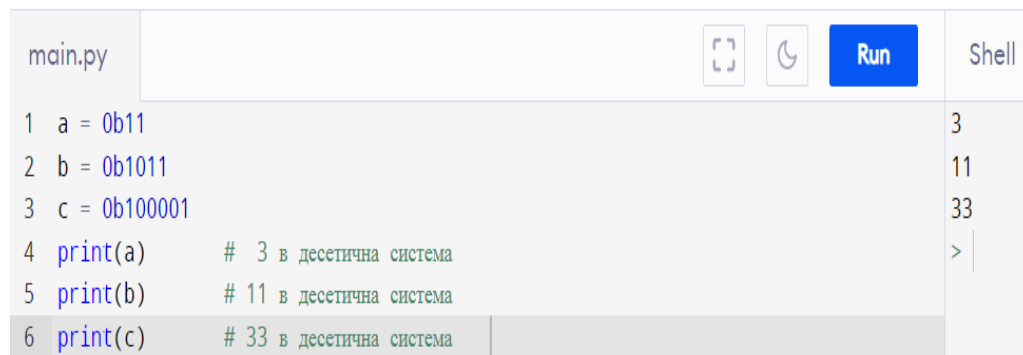
```
age = 21
print("Възраст:", age)      # Възраст: 21
count = 15
print("Количество:", count) # Количество: 15
```

По подразбиране стандартните числа се третираат като десетични числа. Но Python също поддържа числа в двоични, осмични и шестнадесетични системи.

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

А. Представянето на число в **двоична система**, става чрез поставяне на **префикс 0b** пред числото:

```
a = 0b11
b = 0b1011
c = 0b100001
print(a)    # 3 в десетична система
print(b)    # 11 в десетична система
print(c)    # 33 в десетична система
```



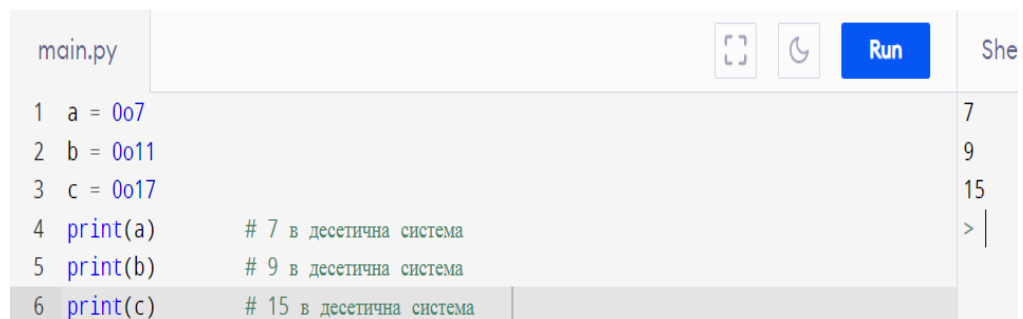
The screenshot shows a Python IDE window titled 'main.py'. The code contains three binary literals: 0b11, 0b1011, and 0b100001. Each is followed by a print statement with a comment indicating its decimal value. The output on the right shows the values 3, 11, and 33. The interface includes a 'Run' button and a 'Shell' tab.

```
main.py
1 a = 0b11
2 b = 0b1011
3 c = 0b100001
4 print(a)    # 3 в десетична система
5 print(b)    # 11 в десетична система
6 print(c)    # 33 в десетична система
```

3
11
33
> |

Б. Представянето на число в **осмична система**, става чрез поставяне на **префикс 0o** пред числото:

```
a = 0o7
b = 0o11
c = 0o17
print(a)    # 7 в десетична система
print(b)    # 9 в десетична система
print(c)    # 15 в десетична система
```



The screenshot shows a Python IDE window titled 'main.py'. The code contains three octal literals: 0o7, 0o11, and 0o17. Each is followed by a print statement with a comment indicating its decimal value. The output on the right shows the values 7, 9, and 15. The interface includes a 'Run' button and a 'She' tab.

```
main.py
1 a = 0o7
2 b = 0o11
3 c = 0o17
4 print(a)    # 7 в десетична система
5 print(b)    # 9 в десетична система
6 print(c)    # 15 в десетична система
```

7
9
15
> |

В. Представянето на число в **шеснадесетична система**, става чрез поставяне на **префикс 0x** пред числото:

a = 0x0A

b = 0xFF

c = 0xA1

print(a) # 10 в десетична система

print(b) # 255 в десетична система

print(c) # 161 в десетична система

```
main.py
1 a = 0x0A
2 b = 0xFF
3 c = 0xA1
4 print(a)      # 10 в десетична система
5 print(b)      # 255 в десетична система
6 print(c)      # 161 в десетична система
```

	Shell
10	10
255	255
161	161
>	>

Трябва да се отбележи, че на каквато и система да се представи число на функцията за печат за извеждане на конзолата, то по подразбиране ще бъде изведено с десетичен знак.

6.3. Дробни числа

Типът **float** представлява число с плаваща запетая, като 1.2 или 34.76. Точката се използва като разделител между цели и дробни части.

height = 1.68

pi = 3.14

weight = 68.

print(height) # 1.68

print(pi) # 3.14

print(weight) # 68.0

```
main.py
1 height = 1.68
2 pi = 3.14
3 weight = 68.
4 print(height) # 1.68
5 print(pi)     # 3.14
6 print(weight) # 68.0
```

	Shell
1.68	1.68
3.14	3.14
68.0	68.0
>	>

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Числото с плаваща запетая може да бъде дефинирано в експоненциална нотация.

Пример:

```
x = 3.9e3
```

```
print(x)                # 3900.0
```

```
x = 3.9e-3
```

```
print(x)                # 0.0039
```

main.py		Run	Shell
1 x = 3.9e3			3900.0
2 print(x)	# 3900.0		0.0039
3 x = 3.9e-3			>
4 print(x)	# 0.0039		

Едно плаващо число може да има само 18 значими знака. Така че в този случай се използват само два знака - 3.9. И ако числото е твърде голямо или твърде малко, тогава може да се запише числото в подобна нотация, като се използва степента. Числото след степенният знак показва степента на числото 10, с което трябва да се умножи основното число - 3.9.

6.4. Комплексни числа

Сложният тип представлява комплексни числа във формат - имагинерната част е последвана от наставката j, истинската_част+мнима_част j

```
complexNumber = 1+2j
```

```
print(complexNumber)    # (1+2j)
```

main.py		Run	Shell
1 complexNumber = 1+2j			(1+2j)
2 print(complexNumber)	# (1+2j)		>

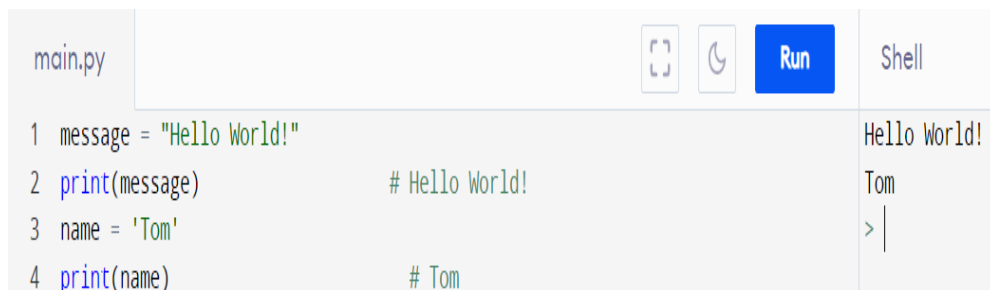
Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Струни

Типът **str** е низ – това са поредица от знаци, затворени в единични или двойни кавички, като "Hello World!" и "Hi!". В Python 3.x низовете представляват набор от символи в Unicode.

Пример:

```
message = "Hello World!"  
print(message)           # Hello World!  
name = 'Tom'  
print(name)              # Tom
```



```
main.py  
1 message = "Hello World!"  
2 print(message)           # Hello World!  
3 name = 'Tom'  
4 print(name)              # Tom
```

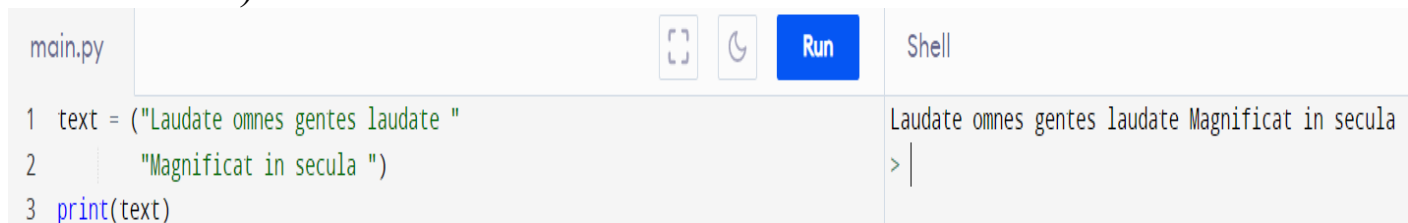
Shell

```
Hello World!  
Tom  
> |
```

Освен това, ако низ има много символи, може да бъде разделен на части и да се постави на различни редове код. В този случай целият низ е затворен в скоби, а отделните му части са в кавички.

Пример:

```
text = ("Laudate omnes gentes laudate "  
        "Magnificat in secula ")  
print(text)
```



```
main.py  
1 text = ("Laudate omnes gentes laudate "  
2         "Magnificat in secula ")  
3 print(text)
```

Shell

```
Laudate omnes gentes laudate Magnificat in secula  
> |
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Ако трябва да се дефинира многоредов текст, тогава текстът се затваря в тройни двойни или единични кавички.

Пример:

"""

Това е коментар

"""

text = "Laudate omnes gentes laudate

Magnificat in secula

Et anima mea laudate

Magnificat in secula

"""

print(text)

main.py	  	Shell
1 '''		Laudate omnes gentes laudate
2 Това е коментар		Magnificat in secula
3 '''		Et anima mea laudate
4 text = '''Laudate omnes gentes laudate		Magnificat in secula
5 Magnificat in secula		
6 Et anima mea laudate		
7 Magnificat in secula		
8 '''		
9 print(text)		>

Когато се използват тройни единични кавички (те не трябва да се бъркат с коментари): **ако текстът в тройни единични кавички е присвоен на променлива, то това е низ, а не коментар.**

Escape последователности в низ

Низът може да съдържа редица специални символи - escape последователности. Някои от тях:

- `\` : позволяват добавяне на наклонена черта вътре в низа
- `'` : позволяват добавяне на единична кавичка вътре в низа
- `"` : позволява добавяне на двойни кавички вътре в низа
- `\n` : преминаване към нов ред
- `\t` : добавяне на раздел (4 отстъпа)

Нека да разгледаме следните няколко последователности.

Пример:

```
message = "Message:\n\"Hello World\""
print(message)
```

Резултат:

```
Message:
"Hello World"
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Въпреки че подобни последователности могат да помогнат в някои случаи, например при поставяне на цитат в низ, да се направи таблица, да се прехвърли на друг ред. Те също могат и да пречат.

Пример:

```
path = "C:\python\name.txt"
print(path)
```

Тук променливата path съдържа някакъв път до файл. Въпреки това, в низа има символи "\n", които ще бъдат интерпретирани като ескаре последователност. И по този начин, се получава КОНЗОЛНИЯ ИЗХОД:

```
c:\python
ame.txt
```

За да се избегнат подобни ситуации, се поставя символ r преди низа.

```
path = r"C:\python\name.txt"
print(path)
```

Вмъкване на стойности в низ

Python позволява вграждане на стойности на други променливи в низ. За целта, вътре в низа, променливите се поставят в къдрави скоби {}, а символът f се поставя преди целия низ:

```
userName = "Tom"
userAge = 37
user = f"name: {userName} age: {userAge}"
print(user)                # name: Tom age: 37
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

В този случай {userName} стойността на променливата userName ще бъде вмъкната на място. По аналогичен начин е {userAge}, вместо, който ще бъде вмъкната стойността на променливата userAge.

Динамично писане

Python е динамично въведен език. Това означава, че **променливата не е строго обвързана с конкретен тип**. Типът на променливата се определя въз основа на присвоената ѝ стойност. Например, когато се присвои низ в двойни или единични кавички, променливата е от тип str. Когато се присвои цяло число, Python автоматично определя типа на променливата като int. За да се дефинира променлива като плаващ обект, ѝ се присвоява дробно число, което и има десетичната запетая. В същото време, в процеса на изпълнение на програмата, може да се промени типа на променливата, като ѝ се присвои стойност от друг тип:

```
userId = "abc"           # тип str
print(userId)
userId = 234              # тип int
print(userId)
```

main.py		Run	Shell
1	userId = "abc" # тип str		abc
2	print(userId)		234
3	userId = 234 # тип int		>
4	print(userId)		

С помощта на вградената функция type() може динамично да се разбере текущия тип на променливата:

```
userId = "abc"           # тип str
print(type(userId))      # <class 'str'>
userId = 234              # тип int
print(type(userId))      # <клас 'int'>
```

main.py		Run	Shell
1	userId = "abc" # тип str		<class 'str'>
2	print(type(userId)) # <class 'str'>		<class 'int'>
3	userId = 234 # тип int		>
4	print(type(userId)) # <клас 'int'>		

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Вход и изход в конзолата

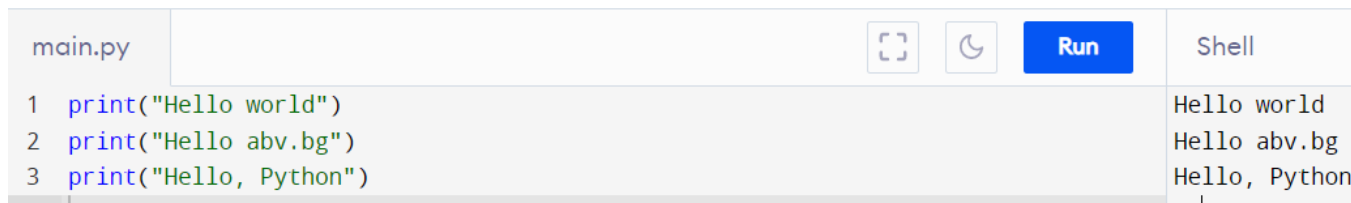
Изход в конзолата

Вградената функция `print()` се използва за отпечатване на информация към конзолата. Когато тази функция се извика, изходната стойност ѝ се предава в скоби:

```
print("Hello abv.bg")
```

Този код ще отпечата низа "Hello abv.bg" в конзолата. Отличителна черта на тази функция е, че по подразбиране тя извежда стойността на отделен ред. Например:

```
print("Hello world")  
print("Hello abv.bg")  
print("Hello, Python")
```



main.py	Run	Shell
1 print("Hello world")		Hello world
2 print("Hello abv.bg")		Hello abv.bg
3 print("Hello, Python")		Hello, Python

Тук три извиквания на функции `print()` извеждат някакво съобщение. Освен това, при извеждане в конзолата, всяко съобщение ще бъде поставено на отделен ред:

```
Hello world  
Hello abv.bg  
Hello Python
```

Това поведение не винаги е удобно. Например, искаме всички стойности да се показват на един ред. За да се направи това, трябва да персонализира поведението на функцията, като се използва крайния параметър.

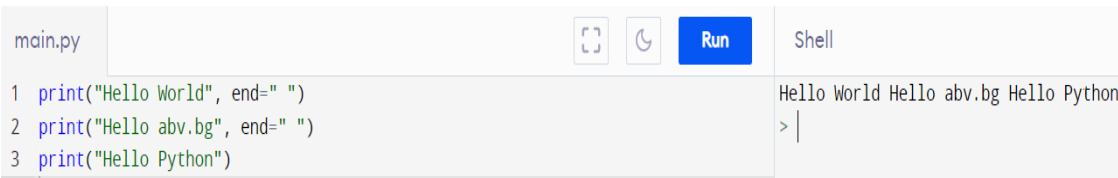
Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Тази опция определя знаците, които се добавят в края на изходния низ. Когато се използва крайния параметър, извикването на функцията `print()` изглежда така:

`print(стойност, end = символ_за_край)`

По подразбиране той е `end` и е равен на символа `"\n"`, който определя прехода на следващия ред. Ето защо функцията за печат отпечатва предадената ѝ стойност на отделен ред по подразбиране. Нека да се създаде функция, която да не поставя стойността на следващия ред, а да я показва на същия ред:

```
print("Hello World", end=" ")
print("Hello abv.bg", end=" ")
print("Hello Python")
```



```
main.py
1 print("Hello World", end=" ")
2 print("Hello abv.bg", end=" ")
3 print("Hello Python")
```

Shell

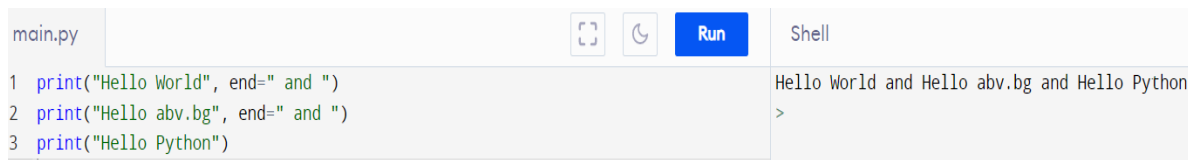
Hello World Hello abv.bg Hello Python

т.е. така изходните стойности ще бъдат разделени с интервал:

Hello World Hello abv.bg Hello Python

И може да бъде не един знак, а набор от знаци:

```
print("Hello World", end=" and ")
print("Hello abv.bg", end=" and ")
print("Hello Python")
```



```
main.py
1 print("Hello World", end=" and ")
2 print("Hello abv.bg", end=" and ")
3 print("Hello Python")
```

Shell

Hello World and Hello abv.bg and Hello Python

В този случай показаните съобщения ще бъдат разделени със знаци `" and "`:

Hello World and Hello abv.bg and Hello Python

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Конзолен вход

Заедно с изхода към конзолата, може да се получи потребителски вход от конзолата, да се получат входни данни. За да направи това, Python дефинира функцията `input()`. На тази функция се предава подкана за въвеждане. И може да се съхрани входния резултат в променлива. Например, нека се дефинира код, за който потребителят да въведе име:

```
name = input("Въведете вашето име: ")
```

```
print(f"Вашето име: {name}")
```

В този случай `input()` на функцията се предава подкана под формата на низ "Въведете вашето име:". Резултат от тази функция е резултатът от въвеждането на потребителя и се предава на променливата `name`. След това може да се отпечата стойността на тази променлива в конзолата с помощта на `print()`.

Пример:

Въведете вашето име: Евгени

Вашето име: Евгени

Пример с множество записи:

```
name = input("Your name: ")
```

```
age = input("Your age: ")
```

```
print(f"Name: {name} Age: {age}")
```

Пример за работата на програмата:

Your name: Tom

Your age: 37

Name: Tom Age: 37

main.py	  	Shell
<pre>1 name = input("Въведете вашето име: ") 2 print(f"Вашето име: {name}")</pre>		Въведете вашето име: Евгени

main.py	  	Shell
<pre>1 name = input("Your name: ") 2 age = input("Your age: ") 3 print(f"Name: {name} Age: {age}")</pre>		Your name: Tom Your age: 37 Name: Tom Age: 37

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Трябва да се има предвид, че всички въведени стойности се третираат като стойности от тип `str`, т.е. низове. И дори да се въведе число, както е във втория случай в кода, то Python пак ще третира въведената стойност като низ, а не като число.

Аритметични операции с числа

Python поддържа всички често срещани аритметични операции:

- `+`

Събиране на две числа:

```
print(6 + 2)      # 8
```

- `-`

Изваждане на две числа:

```
print(6 - 2)      # 4
```

- `*`

Умножение на две числа:

```
print(6 * 2)      # 12
```

- `/`

Деление на две числа:

```
print(6 / 2)      # 3.0
```

- `//`

Целочислено деление на две числа:

```
print(7 / 2)      # 3.5
```

```
print(7 // 2)     # 3
```

Тази операция връща целочисления резултат от деленето, отхвърляйки дробната част

main.py		Run	Shell
1	<code>print(6 + 2)</code>		8
2	<code>print(6 - 2)</code>		4
3	<code>print(6 * 2)</code>		12
4	<code>print(7 / 2)</code>		3.5
5	<code>print(6 ** 2)</code>		36
6	<code>print(7 // 2)</code>		3
7	<code>print(7 % 2)</code>		1

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

- **

Степенуване:

```
1 print(6 ** 2)          # Повдига числото 6 на степен на 2. Резултатът е 36
```

- %

Получаване на остатъка от деление:

```
1 print(7 % 2)           # Връща остатък от разделяне на числото 7 на 2.  
Резултатът е 1
```

В този случай най-близкото число до 7, което се дели на 2 без остатък, е 6. Следователно остатъкът от делението е $7 - 6 = 1$

Когато няколко аритметични операции се използват последователно, те се изпълняват в съответствие с техния приоритет. Първо се извършват операции с по-висок приоритет. Приоритетите на операциите в низходящ ред са показани в следната таблица.

Таблица 1. Приоритети на операциите в низходящ ред в Python

Операции	Посока
**	от дясно на ляво
* / // %	от ляво на дясно
+ -	от ляво на дясно

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Нека да разгледаме следния израз:

```
number = 3 + 4 * 5 ** 2 + 7  
print(number)                                #110
```

Тук повдигането на степен ($5 ** 2$) се изпълнява първо, като операция с по-висок приоритет, след това резултатът се умножава по 4 ($25 * 4$), след това се получава събиране ($3 + 100$) и след това отново събиране ($103 + 7$).

Скоби могат да се използват за предефиниране на реда на операциите (резултатът ще е друг)

```
number = (3 + 4) * (5 ** 2 + 7)  
print(number)                                #224
```

Трябва да се отбележи, че в аритметичните операции могат да участват както цели, така и дробни числа. Ако цяло число (int) и число с плаваща запетая (float) участват в една и съща операция, тогава цялото число се прехвърля към типа float.

main.py	  	Shell
1	number = 3 + 4 * 5 ** 2 + 7	110
2	print(number)	224
3	number = (3 + 4) * (5 ** 2 + 7)	>
4	print(number)	

Аритметични операции със присвояване

Редица специални операции позволяват да присвояване на резултата от операция на първия операнд:

+=

Присвояване на резултата от събиране

-=

Присвояване на резултата от изваждане

***=**

Присвояване на резултата от умножение

/=

Присвояване на резултата от разделяне

//=

Присвояване на резултата от целочислено деление

****=**

Присвояване на степен на число

%=

Присвояване на остатък от деление

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Примери за операции:

```
number = 10
number += 5
print(number)
number -= 3
print(number)
number *= 4
print(number)
```

main.py		Run	Shell
1	number = 10		15
2	number += 5		12
3	print(number)	# 15	48
4	number -= 3		>
5	print(number)	# 12	
6	number *= 4		
7	print(number)	# 48	

Закръгляване и функция round

При извършване на операции с числа с плаваща запетая, трябва да се има предвид, че резултатът от операциите с тях може да не е напълно точен.

Пример:

```
first_number = 2.0001
second_number = 5
third_number = first_number / second_number
print(third_number) # 0.400020000000000004
```

main.py		Run	Shell
1	first_number = 2.0001		0.400020000000000004
2	second_number = 5		>
3	third_number = first_number / second_number		
4	print(third_number)		

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

В този случай се очаква резултатът да е числото 0,40002, но в действителност, чрез поредица от нули, се появява някаква четворка. Или друг израз:

```
print(2.0001 + 0.1)          # 2.10010000000000003
```

В случая по-горе може да се използва вградената функция `round()`, за да се закръгли резултата:

main.py	Run	Shell
<pre>1 print(2.0001 + 0.1) # 2.10010000000000003 2 first_number = 2.0001 3 second_number = 0.1 4 third_number = first_number + second_number 5 print(round(third_number)) # 2 6 first_number = 2.0001 7 second_number = 0.1 8 third_number = first_number + second_number 9 print(round(third_number, 4)) # 2.1001</pre>		<pre>2.10010000000000003 2 2.1001 > </pre>

`round()` числото, което трябва да се закръгли, се предава на функцията. Ако на функцията се предаде едно число, както в примера по-горе, то се закръглява до цяло число. Функцията `round()` може също да приеме второ число, което указва колко десетични знака трябва да съдържа полученото число:

```
first_number = 2.0001
second_number = 0.1
third_number = first_number + second_number
print(round(third_number, 4))    # 2.1001
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

В този случай числото `third_number` се закръгля до 4 знака след десетичната запетая. Ако на функцията се подава само една стойност т.е. само числото, което трябва да се закръгли, то се закръгля до най-близкото цяло число.

Примери за закръгляне:

закръгляне до цяло число

```
print(round(2.49))
```

2 – закръгляне до най-близкото цяло

число в случая 2

```
print(round(2.51))
```

3

Въпреки това, **ако частта, която трябва да се закръгли, е на същото разстояние от две цели числа, тогава закръгляването отива до най-близкото четно число:**

```
print(round(2.5))
```

2 – до най-близкото четно число

```
print(round(3.5))
```

4 – до най-близкото четно число

Закръглянето се извършва до най-близкоторатно на 10 до степента минус частта, която трябва да се закръгли:

закръгляне до два знака след запетая

```
print(round(2.554, 2))
```

2.55

```
print(round(2.5551, 2))
```

2.56

```
print(round(2.554999, 2))
```

2.55

```
print(round(2.499, 2))
```

2.5

main.py

```
1 print(round(2.49))
2 print(round(2.51))
3 print(round(2.5))
4 print(round(3.5))
5 print(round(2.554, 2))
6 print(round(2.5551, 2))
7 print(round(2.554999, 2))
8 print(round(2.499, 2))
```

```
2
3
2
4
2.55
2.56
2.55
2.5
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Имайте предвид обаче, че функцията `round()` не е перфектен инструмент. Например, по-горе, когато се закръглят до цели числа, се прилага правилото, че ако частта, която трябва да се закръгли, е на едно и също разстояние от две стойности, тогава закръгляването се извършва до най-близката четна стойност. В Python, поради факта, че десетичната част от число не може да бъде точно представена като число с плаваща стойност, това може да доведе до някои не съвсем очаквани резултати.

Пример:

закръгляне до два знака след десетичната запетая

```
print(round(2.545, 2))      # 2.54
print(round(2.555, 2))      # 2.56 – закръгляне до четно число
print(round(2.565, 2))      # 2.56
print(round(2.575, 2))      # 2.58
print(round(2.655, 2))      # 2.65 – закръгляне до четно число ???
print(round(2.665, 2))      # 2.67
print(round(2.675, 2))      # 2.67
```

main.py		Run	Shell
1	<code>print(round(2.545, 2))</code>	# 2.54	2.54
2	<code>print(round(2.555, 2))</code>	# 2.56	2.56
3	<code>print(round(2.565, 2))</code>	# 2.56	2.56
4	<code>print(round(2.575, 2))</code>	# 2.58	2.58
5	<code>print(round(2.655, 2))</code>	# 2.65	2.65
6	<code>print(round(2.665, 2))</code>	# 2.67	2.67
7	<code>print(round(2.675, 2))</code>	# 2.67	2.67

Побитови операции с числа

Побитовите операции представляват специален клас операции. Те се изпълняват върху отделни цифри на числа от тип `int`. Но за да се разбере естеството на побитовите операции, трябва да се разбере какво представлява числото като цяло в двоично представяне.

Двоично представяне на числата

В двоичната система всяка цифра от число може да има само две стойности - 0 и 1. Например, 0 с десетичната запетая също ще бъде равно на 0 в двоична система, а 1 с десетичната запетая ще съответства на 1 в двоична. Следващото число в десетичната система - 2 в двоичната система ще отговаря на 10. Тоест, когато добавим 1 към 1, резултатът ще бъде 10. И така нататък. Например, 5 в двоично число е 101 и има три цифри. За да изведете десетично число в двоичната система, можете да използвате спецификатора `0b`:

```
number = 5                                # в двоична форма 101
print(f'number = {number:0b}')            # number = 101
```

main.py				Run	Shell
1	number = 5				number = 101
2	print(f'number = {number:0b}')				>

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Без да посочва спецификатор, функцията `print()` извежда число в десетичната система. В същото време Python позволява незабавно да определите число в двоична форма. За да направите това, числото в двоична форма се посочва след префикса `0b`:

<code>number = 0b101</code>	<code># определя числото в двоична форма</code>
<code>print(f'number = {number:0b}')</code>	<code># number = 101</code>
<code>print(f'number = {number}')</code>	<code># number = 5 – в десетична система</code>

Още няколко примера за преобразуване на числа от двоична в десетична система:

```
number1 = 1
number2 = 2
number3 = 3
number4 = 4
number5 = 5
number6 = 6
```

main.py	Run	Shell
<pre>1 number = 0b101 2 print(f"number = {number:0b}") 3 print(f"number = {number}") 4 number1 = 1 5 number2 = 2 6 number3 = 3 7 number4 = 4 8 number5 = 5 9 number6 = 6 10 print(f"number1 = {number1:0b}") 11 print(f"number2 = {number2:0b}") 12 print(f"number3 = {number3:0b}") 13 print(f"number4 = {number4:0b}") 14 print(f"number5 = {number5:0b}") 15 print(f"number6 = {number6:0b}")</pre>		<pre>number = 101 number = 5 number1 = 1 number2 = 10 number3 = 11 number4 = 100 number5 = 101 number6 = 110 > </pre>

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Булеви операции

Логическите операции се извършват върху отделни цифри на число. Python има следните логически операции:

- & (булево умножение)

Умножението се извършва бит по бит и ако и двата операнда имат битови стойности, равни на 1, тогава операцията връща 1, в противен случай се връща числото 0. Например:

```
x1 = 2          #010
y1 = 5          #101
z1 = x1 & y1
print(f'z1 = {z1}')    # z1 = 0
x2 = 4          #100
y2 = 5          #101
z2 = x2 & y2
print(f'z2 = {z2}')    # z2 = 4
print(f'z2 = {z2:0b}') # z2 = 100
```

main.py		Shell
1 x1 = 2	#010	z1 = 0
2 y1 = 5	#101	z2 = 4
3 z1 = x1 & y1		z2 = 100
4 print(f'z1 = {z1}')	# z1 = 0	>
5 x2 = 4	#100	
6 y2 = 5	#101	
7 z2 = x2 & y2		
8 print(f'z2 = {z2}')	#z2 = 4	
9 print(f'z2 = {z2:0b}')	#z2 = 100	

В първия случай се разглеждат две числа 2 и 5, като 2 в двоичен код представлява числото 010, а 5 съответно е 101. Ако умножим числата бит по бит ($0 * 1, 1 * 0, 0 * 1$) ще се получи резултат 000.

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Във втория случай, вместо 2, има число 4, което има 1 в първата цифра, точно като числото 5, така че в крайна сметка ще се получи $(1 * 1, 0 * 0, 0 * 1) = 100$, т.е. числото 4 в десетичен формат.

в двоична система 0b110

- | (Логическо събиране)

Подобно на логическото умножение, операцията се извършва и върху двоични цифри, но сега се връща единица, ако поне едно число в този бит има едно.

Пример:

x1 = 2	#010
y1 = 5	#101
z1 = x1 y1	# 111
print(f"z1 = {z1}")	# z1 = 7
print(f"z1 = {z1:0b}")	# z1 = 111
x2 = 4	#100
y2 = 5	#101
z2 = x2 y2	#101
print(f"z2 = {z2}")	# z2 = 5
print(f"z2 = {z2:0b}")	# z2 = 101

1 x1 = 2	#010	z1 = 7
2 y1 = 5	#101	z1 = 111
3 z1 = x1 y1	# 111	z2 = 5
4 print(f"z1 = {z1}")	# z1 = 7	z2 = 101
5 print(f"z1 = {z1:0b}")	# z1 = 111	>
6 x2 = 4	#100	
7 y2 = 5	#101	
8 z2 = x2 y2	#101	
9 print(f"z2 = {z2}")	# z2 = 5	
10 print(f"z2 = {z2:0b}")	# z2 = 101	

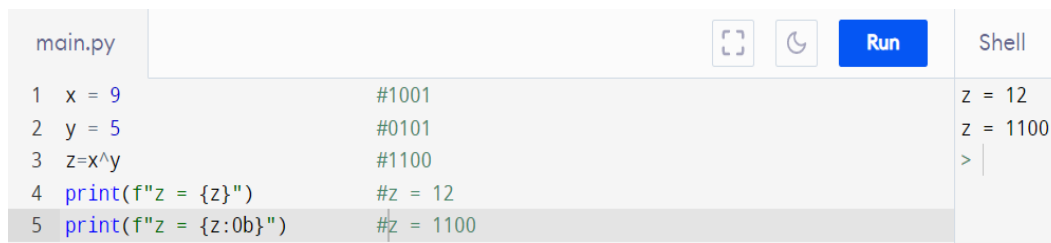
Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

- $\wedge$ (логическо XOR)

Ако стойностите на текущия бит и за двете числа са различни, тогава се връща 1, в противен случай се връща 0. Тази операция се нарича още XOR.

Пример:

<code>x = 9</code>	<code>#1001</code>
<code>y = 5</code>	<code>#0101</code>
<code>z=x^y</code>	<code>#1100</code>
<code>print(f"z = {z}")</code>	<code># z = 12</code>
<code>print(f"z = {z:0b}")</code>	<code># z = 1100</code>



```
main.py
1 x = 9          #1001
2 y = 5          #0101
3 z=x^y          #1100
4 print(f"z = {z}")    #z = 12
5 print(f"z = {z:0b}") #z = 1100
```

Shell

```
z = 12
z = 1100
> |
```

Тук числото 9 в двоична форма е 1001. Числото 5 е 0101. Операцията XOR дава следния резултат: $1 \wedge 0$, $0 \wedge 1$, $0 \wedge 0$, $1 \wedge 1$. Тук се вижда, че първите две цифри на числата съдържат различни стойности, така че първите две цифри ще получат стойност 1. А последните две цифри на числата съдържат едни и същи стойности, така че последните две цифри ще получат стойност 0. Така се получава числото 1100 или 12 в десетична бройна система. Често тази операция се използва за просто криптиране:

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

```
x = 45
key = 102
encrypt = x ^ key
```

```
print(f"Шифровано число: {encrypt}")
decrypt = encrypt ^ key
print(f"Декриптиран номер: {decrypt}")
```

Можете също да използвате тази операция за обмен на стойностите на числата:

```
x = 9          #1001
y = 5          #0101
x=x^y
y=x^y
x=x^y
print(f"x = {x}")    # x = 5
print(f"y = {y}")    # y = 9
• ~ (обратно)
```

Обръща числото. Изразът $\sim x$ всъщност е същият $-(x+1)$.

Пример:

```
x = 5
y = ~x;
print(f"y: {y}")    # -6
```

```
main.py
1 x = 45          # Стойност за криптиране - в двоична форма 101101
2 key = 102       # Ако ключът в двоична форма е 11001110
3 encrypt = x ^ key # Резултатът ще бъде числото 1001011 или 75
4               # Резултатът ще бъде числото 45
5 print(f"Шифровано число: {encrypt}")
6 decrypt = encrypt ^ key
7 print(f"Декриптиран номер: {decrypt}")
```

```
Shell
Шифровано число: 75
Декриптиран номер: 45
> |
```

```
main.py
1 x = 9          #1001
2 y = 5          #0101
3 x=x^y
4 y=x^y
5 x=x^y
6 print(f"x = {x}")    #x = 5
7 print(f"y = {y}")    #y = 9
```

```
Shell
x = 5
y = 9
> |
```

```
main.py
1 x = 5
2 y = ~x;
3 print(f"y: {y}")    # -6
```

```
Shell
y: -6
> |
```

Операции за смяна

Операциите за смяна се извършват и върху цифри от числа. Изместването може да се случи надясно и наляво.

- $x \ll y$ - измества числото x наляво с y цифри. Например, $4 \ll 1$ измества числото 4 (което е 100 в двоично число) с една цифра наляво, което води до 1000 или 8 в десетична система.
- $x \gg y$ - измества числото x надясно с y цифри. Например, $16 \gg 1$ измества числото 16 (което е 10 000 в двоично число) с една цифра вдясно, което води до 1000 или 8 в десетична система.

По този начин, ако първоначалното число, което трябва да бъде изместено в една или друга посока, се раздели на две, тогава всъщност се получава умножение или делене на две. Следователно подобна операция може да се използва вместо директно умножение или деление на две. Например:

$a = 16$ # в двоична форма 10000

$b = 2$

$c = a \ll b$ # Изместване 10000 наляво с 2 цифри, равно на 1 000 000 или 64 в десетична система

`print(c)` #64

$d = a \gg b$ #Преместване на числото 10 000 надясно с 2 цифри, равно на 100 или 4 в десетична система

`print(d)` #4

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

В този случай числата, които участват в операциите, не трябва да са кратни на 2:

`a = 22` # в двоична форма 10110

`b = 2`

`c = a << b` # Изместване 10110 наляво с 2 цифри, равно на 1011000 или 88 в десетична система

`print (c)` #88

`d = a >> b` # Shift 10110 надясно 2 цифри, равно на 101 или 5 в десетична система

`print (d)` #5

main.py	Run	Shell
<pre>1 a = 16 2 b = 2 3 c = a << b 4 print(c) 5 d = a >> b 6 print(d) 7 a = 22 8 b = 2 9 c = a << b 10 print (c) 11 d = a >> b 12 print (d)</pre>		<pre>64 4 88 5 > </pre>

Условни изрази

Редица операции представляват условни изрази. Всички тези операции приемат два операнда и връщат булева стойност, която в Python представлява типа bool. Има само две булеви стойности - True (изразът е true) и False (изразът е false).

Операции за сравнение

Най-простите условни изрази са операции за сравнение, които сравняват две стойности. Python поддържа следните операции за сравнение:

- ==

Връща True, ако и двата операнда са равни. В противен случай връща False.

- !=

Връща True, ако и двата операнда НЕ са равни. В противен случай връща False.

- > (по-голямо от)

Връща True, ако първият операнд е по-голям от втория.

- < (по-малко от)

Връща True, ако първият операнд е по-малък от втория.

- >= (по-голямо или равно на)

Връща True, ако първият операнд е по-голям или равен на втория.

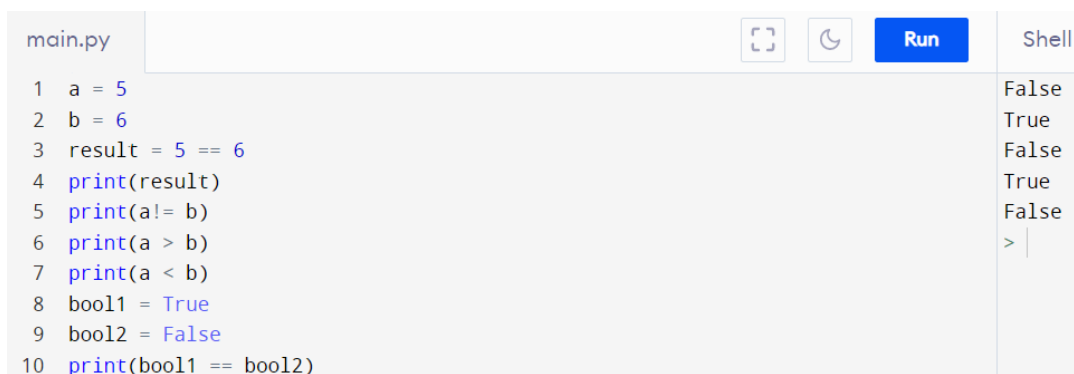
- <= (по-малко или равно на)

Връща True, ако първият операнд е по-малък или равен на втория.

Примери за операции за сравнение:

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

```
a = 5
b = 6
result = 5 == 6      # запазване на резултата от операцията в променлива
print(result)        # False - 5 не е равно на 6
print(a != b)         # True
print(a > b)          # False - 5 е по-малко от 6
print(a < b)          # True
bool1 = True
bool2 = False
print(bool1 == bool2) # False - bool1 не е равен на bool2
```



The screenshot shows a code editor with a file named 'main.py'. The code is as follows:

```
1 a = 5
2 b = 6
3 result = 5 == 6
4 print(result)
5 print(a != b)
6 print(a > b)
7 print(a < b)
8 bool1 = True
9 bool2 = False
10 print(bool1 == bool2)
```

On the right side, there is a 'Shell' window showing the output of the program:

```
False
True
False
True
True
False
> |
```

Операторите за сравнение могат да сравняват различни обекти - низове, числа, логически стойности, но и двата операнда на операцията трябва да представляват един и същ тип.

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Булеви операции

Логическите операции се използват за създаване на сложни условни изрази.

Python има следните логически оператори:

- and (логическо умножение)

Връща True, ако и двата изказа са True

```
age = 22
```

```
weight = 58
```

```
result = age > 21 and weight == 58
```

```
print(result)           # True
```

В този случай операторът and сравнява резултатите от два изказа: `age > 21` с `weight == 58`. И ако и двата изказа връщат True, тогава операторът and също връща True. Освен това един от изразите не действа непременно като операция за сравнение: може да бъде друга логическа операция или просто булева променлива, която съхранява вярно или невярно.

```
age = 22
```

```
weight = 58
```

```
isMarried = False
```

```
result = age > 21 and weight == 58 and isMarried
```

```
print(result)           # False, тъй като isMarried = False
```

```
main.py  [ ] [ ] Run Shell
1 age = 22
2 weight = 58
3 result = age > 21 and weight == 58
4 print(result)
True
>
```

```
main.py  [ ] [ ] Run Shell
1 age = 22
2 weight = 58
3 isMarried = False
4 result = age > 21 and weight == 58 and isMarried
5 print(result)
False
>
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

- **or** (логично допълнение)

Връща True, ако поне един от изразите е True

```
age = 22
```

```
isMarried = False
```

```
result = age > 21 or isMarried
```

```
print(result)
```

True, тъй като age > 21 равно True

main.py	  	Shell
<pre>1 age = 22 2 isMarried = False 3 result = age > 21 or isMarried 4 print(result)</pre>		<pre>True > </pre>

- **not** (логическо отрицание)

Връща True, ако изразът е False

```
age = 22
```

```
isMarried = False
```

```
print(not age > 21)
```

False

```
print(not isMarried)
```

True

main.py	  	Shell
<pre>1 age = 22 2 isMarried = False 3 print(not age > 21) 4 print(not isMarried)</pre>		<pre>False True > </pre>

Ако един от операндите на оператора and върне False, тогава другият операнд вече не се оценява, тъй като операторът така или иначе ще върне False. Това поведение позволява леко да увеличим производителността, тъй като не е нужно да се харчат ресурси за оценка на вторият операнд. По същия начин, ако един от операндите на оператора или върне True, тогава вторият операнд не се оценява, тъй като операторът така или иначе ще върне True.

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Оператор in

Операторът `in` връща `True`, ако има определена стойност в някакъв набор от стойности.

Има следната форма: **стойност in набор_от_стойности**

Например низ представлява набор от знаци. И с помощта на оператор `in` може да се провери дали съдържа някакъв подниз:

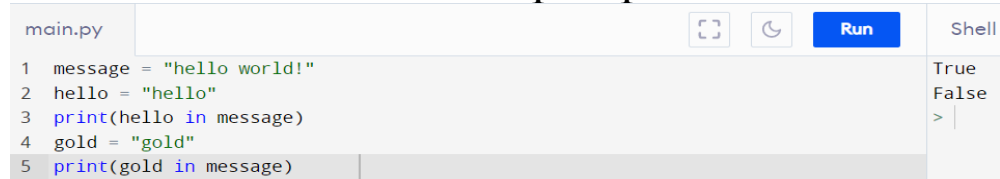
```
message = "hello world!"
```

```
hello = "hello"
```

```
print(hello in message)    # True - подниз hello в низ "hello world!"
```

```
gold = "gold"
```

```
print(gold in message)    # False – подниза "gold" не е в низ "hello world!,,
```



```
main.py
1 message = "hello world!"
2 hello = "hello"
3 print(hello in message)
4 gold = "gold"
5 print(gold in message)
```

Shell
True
False
>

Ако трябва да се провери дали има стойност в набора от стойности, тогава може да се използва модификацията на оператора - **not in**. Връща се `True`, ако **НЯМА** дефинирана стойност в набора от стойности:

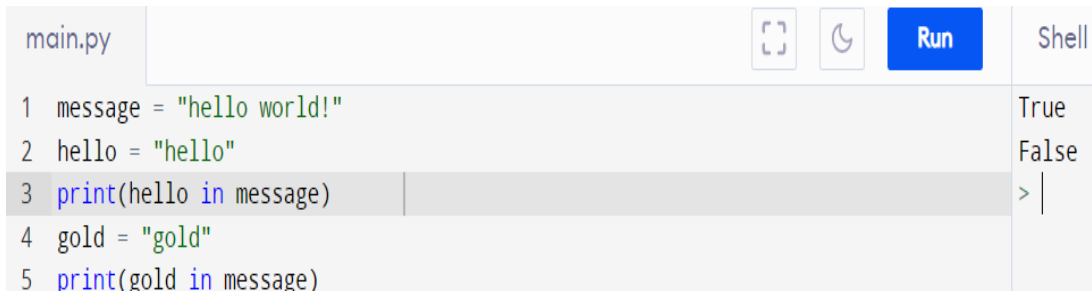
```
message = "hello world!"
```

```
hello = "hello"
```

```
print(hello not in message)    # False
```

```
злато = " gold"
```

```
печат (gold not in message)    # True
```



```
main.py
1 message = "hello world!"
2 hello = "hello"
3 print(hello in message)
4 gold = "gold"
5 print(gold in message)
```

Shell
True
False
>

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Условен оператор if

Условните конструкции използват условни изрази и в зависимост от тяхната стойност насочват изпълнението на програмата по един от пътищата. Една такава конструкция е конструкцията if. Има следното формално определение:

if булев_израз:

инструкции

[elif булев израз:

инструкции]

[else:

инструкции]

В най-простата си форма ключовата дума if е последвана от булев израз. И ако този логически израз върне True, тогава се изпълнява следният блок от инструкции, всеки от които трябва да започне на нов ред и трябва да бъде с отстъп от началото на израза if (желателно е да се отстъпят 4 интервала или броят на интервалите, което е кратно на 4):

```
language = "english"
```

```
if language == " english ":
```

```
    print("Hello")
```

```
print("End")
```

main.py

```
1 language = "english"
2 if language == " english ":
3     print("Hello")
4 print("End")
```



Run

Shell

End

>

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Тъй като в този случай стойността на езиковата променлива е "english", ще се изпълни блокът if, който съдържа само един израз - `print("Hello")`. В резултат на това конзолата ще покаже следните редове:

Hello

End

Обърнете внимание на последния ред в кода, който показва съобщението "End". Той не е с отстъп от началото на реда, така че не принадлежи на блока if и така или иначе ще бъде изпълнен, дори ако изразът в конструкцията if върне False. Но ако се добави с отстъп, той също ще принадлежи към конструкцията if:

```
language = "english"
```

```
if language == "english":
```

```
    print("Hello")
```

```
    print("End")
```

Блок else

Ако трябва да се дефинира алтернативно решение в случай, че изразът if върне False, то може да се използва блока else:

```
language = "russian"
```

```
language == "english":
```

```
    print("Hello")
```

```
else:
```

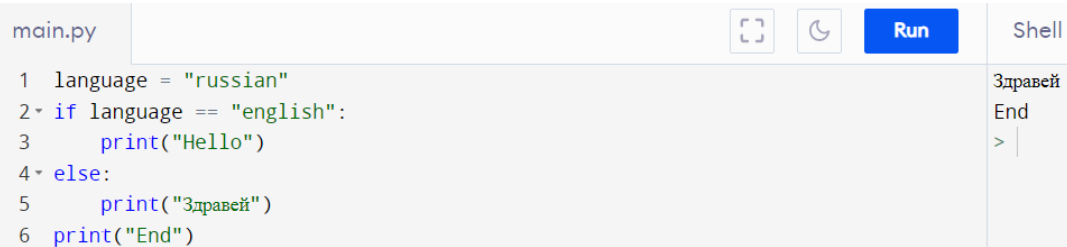
```
    print("Здравей")
```

```
print("End")
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Ако изразът `language == "english"` върне `True`, то блокът `if` се изпълнява, в противен случай се изпълнява блокът `else`. И тъй като в този случай условието `language == "english"` връща `False`, инструкцията от блока ще бъде изпълнена `else` частта. Освен това инструкциите на блока `else` също трябва да са с отстъп от началото на реда. В примера `print("End")` няма отстъп, затова не е включен в блока `else` и ще се изпълни независимо от това на какво е равно условието `language == "english"`. Това означава, че конзолата ще покаже редовете:

Здравей
End



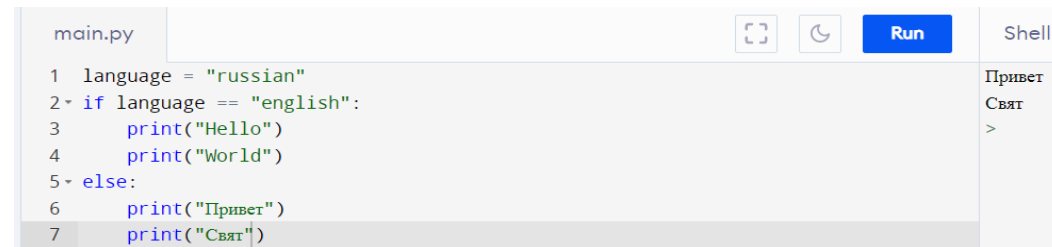
```
main.py
1 language = "russian"
2 if language == "english":
3     print("Hello")
4 else:
5     print("Здравей")
6 print("End")
```

Shell

Здравей
End
>

Блокът `else` може също да има множество изрази, които трябва да бъдат с отстъп от началото на реда:

```
language = "russian"
if language == "english":
    print("Hello")
    print("World")
else:
    print("Привет")
    print("мир")
```



```
main.py
1 language = "russian"
2 if language == "english":
3     print("Hello")
4     print("World")
5 else:
6     print("Привет")
7     print("Свят")
```

Shell

Привет
Свят
>

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

elif

Ако трябва да се въведат няколко алтернативни условия, тогава може да се използват допълнителни elif блокове, последвани от блок от изрази.

```
language = "german"
if language == "english":
    print("Hello")
    print("World")
elif language == "german":
    print("Hallo")
    print("Welt")
else:
    print("Здравей")
    print("свят")
```



```
main.py
1 language = "german"
2 if language == "english":
3     print("Hello")
4     print("World")
5 elif language == "german":
6     print("Hallo")
7     print("Welt")
8 else:
9     print("Здравей")
10    print("свят")
```

Run Shell

Hallo
Welt
>

Python първо проверява израза if. Ако е True, тогава операторите в блока if се изпълняват. Ако това условие върне False, тогава Python проверява израза от elif. Ако изразът след elif е равен True, тогава се изпълняват инструкциите от блока elif. Но ако е равно на False, тогава се изпълняват else инструкциите от блока. Ако е необходимо, можете да се дефинират няколко блока elif за различните условия.

Пример:

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

```
language = "german"
if language == "english":
    print("Hello")
elif language == "german":
    print("Hallo")
elif language == "french":
    print("Salut")
else:
    print("Здравей")
```

Вложени конструкции if

Конструкцията if от своя страна може сама по себе си да е с вложена конструкции if:

```
language = "english"
daytime="morning"
if language == "english":
    print("English")
    if daytime == "morning":
        print("Good morning")
    else:
        print("Good evening")
```

```
main.py
1 language = "german"
2 if language == "english":
3     print("Hello")
4 elif language == "german":
5     print("Hallo")
6 elif language == "french":
7     print("Salut")
8 else:
9     print("Здравей")
```

Run Shell

Hallo
> |

```
main.py
1 language = "english"
2 daytime="morning"
3 if language == "english":
4     print("English")
5     if daytime == "morning":
6         print("Good morning")
7     else:
8         print("Good evening")
```

Run Shell

English
Good morning
> |

Тук конструкцията if съдържа вложена конструкция if/else. Тоест, ако езиковата променлива е равна на "english", тогава вложената конструкция if/else допълнително проверява стойността на променливата за деня - дали е равна на низа "morning" или не. И в този случай ще се получи следния конзолен изход:

```
English
Good morning
```

Тема 2_2. Основи на Python. Променливи и типове данни. Вход и изход. Аритметични операции с числа. Побитови операции с числа. Условни изрази. Условна конструкция if.

Вложените оператори if трябва да бъдат с отстъп, а изразите във вложените конструкции също трябва да бъдат с отстъп. Неправилно поставен отстъп може да промени логиката на програмата. По този начин предишният пример НЕ е същият като следния:

```
language = "english"
daytime = "morning"
if language == "english":
    print("English")
if daytime == "morning":
    print("Good morning")
else:
    print("Good evening")
```

По същия начин, вложени конструкции if/elif/else могат да бъдат поставени в elif и else блокове:

```
language = "russian"
daytime = "morning"
if language == "english":
    if daytime == "morning":
        print("Good morning")
    else:
        print("Good evening")
else:
    if daytime == "morning":
        print("Добро утро")
    else:
        print("Добър вечер ")
```

main.py	  	Shell
<pre>1 language = "russian" 2 daytime = "morning" 3 if language == "english": 4 if daytime == "morning": 5 print("Good morning") 6 else: 7 print("Good evening") 8 else: 9 if daytime == "morning": 10 print("Добро утро") 11 else: 12 print("Добър вечер ")</pre>		Добро утро >

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009